

HW2-Coding-GirardW

October 5, 2026

1 Homework 2 - MTH 602 - Coding Questions

William Girard

Due Date: October 5th, 2026

1.1 Floating Point Numbers

1.1.1 1. (15 points) Consider an N -term Taylor series expansion of the exponential function

$$e^x \approx \sum_{n=0}^{N-1} \frac{x^n}{n!} = \hat{f}_N(x),$$

and the corresponding Python code:

```
def taylor_exp(N, x):  
    """  
    Compute the N-term Taylor series expansion of e^x about 0, evaluated at x.  
  
    Parameters  
    -----  
    N : int  
        Number of terms in the Taylor expansion.  
    x : float  
        Point at which to evaluate the expansion.  
  
    Returns  
    -----  
    float  
        Approximation of e^x using the first N terms of the series.  
    """  
    import math  
  
    total = 0.0  
    for n in range(N):  
        total += (x**n) / math.factorial(n)  
    return total
```

Now consider using this Python function to compute e^x at $x = -20$.

(a) (5 points) Plot the relative error

$$E_N = \frac{|\hat{f}_N(-20) - e^{-20}|}{|e^{-20}|}$$

as a function of x for $x = 1, 2, \dots$. What is the smallest relative error you can achieve? Make the plot using a semilog- scale.

```
[1]: import math
```

```
[2]: def taylor_exp(N, x):  
  
    """  
    Compute the N-term Taylor series expansion of  $e^x$  about 0, evaluated at  $x$ .  
  
    Parameters  
    -----  
    N : int  
    Number of terms in the Taylor expansion.  
    x : float  
    Point at which to evaluate the expansion.  
  
    Returns  
    -----  
    float  
    Approximation of  $e^x$  using the first N terms of the series.  
    """  
  
    total = 0.0  
    for n in range(N):  
        total += (x**n) / math.factorial(n)  
  
    return total
```

```
[3]: # experiment with a few large values of N
```

```
print('N=50', taylor_exp(50, -20))  
print('N=100', taylor_exp(100, -20))  
print('N=200', taylor_exp(200, -20))  
print('N=500', taylor_exp(500, -20))  
print('N=1000', taylor_exp(1000, -20))
```

```
N=50 -2.6549849713380542  
N=100 5.47810291652921e-10  
N=200 5.47810291652921e-10  
N=500 5.47810291652921e-10  
N=1000 5.47810291652921e-10
```

```
[4]: N = 150 # higher N values don't approximate any better (plateau)  
error_list = []  
  
for n in range(1, N+1):
```

```

approx = taylor_exp(n, -20)
true_e = math.exp(-20)
error = (abs(approx - true_e))/(abs(true_e))
error_list.append(error)

```

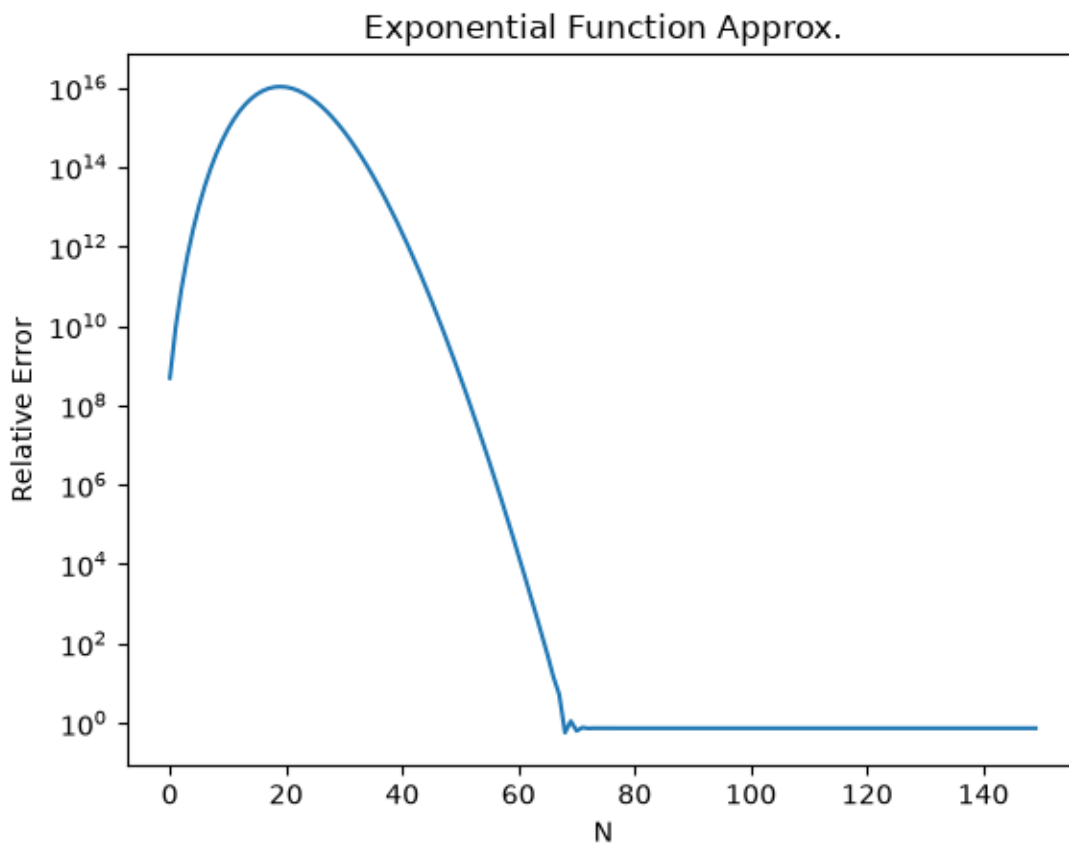
```

[5]: import matplotlib.pyplot as plt

plt.plot(list(range(N)), error_list)
plt.semilogy()
plt.title('Exponential Function Approx.')
plt.xlabel('N')
plt.ylabel('Relative Error')
plt.show()

print(f'\nMinimum error obtained was: {min(error_list)}')

```



Minimum error obtained was: 0.5665545568629835

The relative error starts off very high before dropping down and becoming constant. However, the minimum error of 0.5665545568629835 is much much larger than the expected error of 10^{-16} , so,

clearly something is wrong.

- (b) (5 points) You should find the relative error is much larger than the expected value of $\approx 10^{-16}$. Briefly explain the numerical reason for this behavior (hint: think about cancellation when is negative and large in magnitude).

```
[6]: # print each term in the taylor series
# will help me answer part B by investigating what the terms are actually
    ↪evaluating to

def mod_taylor_exp(N, x):

    """
    Compute the N-term Taylor series expansion of  $e^x$  about 0, evaluated at x.

    Parameters
    -----
    N : int
    Number of terms in the Taylor expansion.
    x : float
    Point at which to evaluate the expansion.

    Returns
    -----
    float
    Approximation of  $e^x$  using the first N terms of the series.
    """

    total = 0.0
    for n in range(N):
        total += (x**n) / math.factorial(n)
        print(f'Current term: {(x**n) / math.factorial(n)} - Running Total:
    ↪{total}')

    return total

total = mod_taylor_exp(100, -20)
print(f'\nTotal: {total}')
```

```
Current term: 1.0 - Running Total: 1.0
Current term: -20.0 - Running Total: -19.0
Current term: 200.0 - Running Total: 181.0
Current term: -1333.3333333333333 - Running Total: -1152.3333333333333
Current term: 6666.666666666667 - Running Total: 5514.3333333333334
Current term: -26666.666666666668 - Running Total: -21152.333333333336
Current term: 88888.88888888889 - Running Total: 67736.55555555556
Current term: -253968.25396825396 - Running Total: -186231.6984126984
Current term: 634920.6349206349 - Running Total: 448688.9365079365
```

Current term: -1410934.7442680777 - Running Total: -962245.8077601411
Current term: 2821869.4885361553 - Running Total: 1859623.6807760142
Current term: -5130671.797338464 - Running Total: -3271048.1165624503
Current term: 8551119.662230773 - Running Total: 5280071.545668323
Current term: -13155568.711124267 - Running Total: -7875497.165455945
Current term: 18793669.58732038 - Running Total: 10918172.421864435
Current term: -25058226.116427176 - Running Total: -14140053.69456274
Current term: 31322782.645533968 - Running Total: 17182728.950971227
Current term: -36850332.524157606 - Running Total: -19667603.57318638
Current term: 40944813.91573068 - Running Total: 21277210.3425443
Current term: -43099804.12182177 - Running Total: -21822593.77927747
Current term: 43099804.12182177 - Running Total: 21277210.3425443
Current term: -41047432.49697311 - Running Total: -19770222.154428814
Current term: 37315847.72452101 - Running Total: 17545625.570092198
Current term: -32448563.23871392 - Running Total: -14902937.668621723
Current term: 27040469.365594935 - Running Total: 12137531.696973212
Current term: -21632375.49247595 - Running Total: -9494843.795502737
Current term: 16640288.840366114 - Running Total: 7145445.044863377
Current term: -12326139.881752677 - Running Total: -5180694.8368893005
Current term: 8804385.62982334 - Running Total: 3623690.7929340396
Current term: -6071990.089533338 - Running Total: -2448299.2965992987
Current term: 4047993.3930222257 - Running Total: 1599694.096422927
Current term: -2611608.6406595004 - Running Total: -1011914.5442365734
Current term: 1632255.4004121877 - Running Total: 620340.8561756143
Current term: -989245.6972195077 - Running Total: -368904.84104389336
Current term: 581909.2336585339 - Running Total: 213004.39261464053
Current term: -332519.5620905908 - Running Total: -119515.1694759503
Current term: 184733.09005032823 - Running Total: 65217.92057437793
Current term: -99855.72435152877 - Running Total: -34637.80377715084
Current term: 52555.64439554146 - Running Total: 17917.84061839062
Current term: -26951.612510534083 - Running Total: -9033.771892143464
Current term: 13475.806255267042 - Running Total: 4442.034363123577
Current term: -6573.564026959532 - Running Total: -2131.529663835955
Current term: 3130.268584266444 - Running Total: 998.7389204304891
Current term: -1455.9388764029973 - Running Total: -457.1999559725082
Current term: 661.7903983649987 - Running Total: 204.59044239249056
Current term: -294.1290659399994 - Running Total: -89.53862354750885
Current term: 127.88220258260844 - Running Total: 38.343579035099594
Current term: -54.41795854579083 - Running Total: -16.074379510691237
Current term: 22.674149394079514 - Running Total: 6.5997698833882765
Current term: -9.25475485472633 - Running Total: -2.6549849713380542
Current term: 3.7019019418905326 - Running Total: 1.0469169705524783
Current term: -1.4517262517217775 - Running Total: -0.40480928116929915
Current term: 0.5583562506622222 - Running Total: 0.153546969492923
Current term: -0.21070047194800834 - Running Total: -0.057153502455085325
Current term: 0.07803721183259568 - Running Total: 0.020883709377510357
Current term: -0.028377167939125703 - Running Total: -0.007493458561615346
Current term: 0.010134702835402036 - Running Total: 0.0026412442737866904

Current term: -0.003556036082597206 - Running Total: -0.0009147918088105154
 Current term: 0.0012262193388266228 - Running Total: 0.00031142753001610736
 Current term: -0.00041566757248360093 - Running Total: -0.00010424004246749357
 Current term: 0.00013855585749453364 - Running Total: 3.4315815027040065e-05
 Current term: -4.542814999820775e-05 - Running Total: -1.1112334971167684e-05
 Current term: 1.4654241934905726e-05 - Running Total: 3.5419069637380417e-06
 Current term: -4.6521402967954685e-06 - Running Total: -1.1102333330574268e-06
 Current term: 1.4537938427485839e-06 - Running Total: 3.4356050969115705e-07
 Current term: -4.473211823841797e-07 - Running Total: -1.0376067269302264e-07
 Current term: 1.3555187344975142e-07 - Running Total: 3.179120075672879e-08
 Current term: -4.0463245805895944e-08 - Running Total: -8.672045049167157e-09
 Current term: 1.1900954648792925e-08 - Running Total: 3.2289095996257682e-09
 Current term: -3.4495520721138913e-09 - Running Total: -2.206424724881231e-10
 Current term: 9.855863063182547e-10 - Running Total: 7.649438338301316e-10
 Current term: -2.7762994544176187e-10 - Running Total: 4.873138883883697e-10
 Current term: 7.71194292893783e-11 - Running Total: 5.64433317677748e-10
 Current term: -2.112861076421323e-11 - Running Total: 5.433047069135347e-10
 Current term: 5.710435341679252e-12 - Running Total: 5.490151422552139e-10
 Current term: -1.5227827577811338e-12 - Running Total: 5.474923594974328e-10
 Current term: 4.007323046792458e-13 - Running Total: 5.47893091802112e-10
 Current term: -1.0408631290370019e-13 - Running Total: 5.477890054892083e-10
 Current term: 2.668879818043595e-14 - Running Total: 5.478156942873888e-10
 Current term: -6.756657767198974e-15 - Running Total: 5.478089376296215e-10
 Current term: 1.6891644417997436e-15 - Running Total: 5.478106267940633e-10
 Current term: -4.1707763995055395e-16 - Running Total: 5.478102097164234e-10
 Current term: 1.0172625364647658e-16 - Running Total: 5.47810311442677e-10
 Current term: -2.4512350276259414e-17 - Running Total: 5.478102869303268e-10
 Current term: 5.836273875299861e-18 - Running Total: 5.478102927666006e-10
 Current term: -1.3732409118352614e-18 - Running Total: 5.478102913933597e-10
 Current term: 3.1935835158959566e-19 - Running Total: 5.47810291712718e-10
 Current term: -7.341571300910245e-20 - Running Total: 5.478102916393023e-10
 Current term: 1.6685389320250557e-20 - Running Total: 5.478102916559877e-10
 Current term: -3.749525689943946e-21 - Running Total: 5.478102916522382e-10
 Current term: 8.332279310986546e-22 - Running Total: 5.478102916530714e-10
 Current term: -1.8312701782388013e-22 - Running Total: 5.478102916528883e-10
 Current term: 3.98102212660609e-23 - Running Total: 5.478102916529281e-10
 Current term: -8.561337906679762e-24 - Running Total: 5.478102916529195e-10
 Current term: 1.821561256740375e-24 - Running Total: 5.478102916529214e-10
 Current term: -3.8348658036639477e-25 - Running Total: 5.478102916529209e-10
 Current term: 7.989303757633224e-26 - Running Total: 5.47810291652921e-10
 Current term: -1.6472791252852008e-26 - Running Total: 5.47810291652921e-10
 Current term: 3.3617941332351035e-27 - Running Total: 5.47810291652921e-10
 Current term: -6.7915032994648555e-28 - Running Total: 5.47810291652921e-10

Total: 5.47810291652921e-10

The above printed lines reveal the reason why the actual error is so much higher than the expected error. The term is constantly being added to negative numbers. Therefore, two problems with

computer arithmetic are coming together to cause a huge problem: rounding errors and catastrophic cancellation.

- 1) Rounding errors are occurring due to the limited precision of computers. Decimal numbers may be unbound, but a computer can only store the integer result up to a certain number of bits (such as floating-point 32 or 64). Normally, this isn't too much of a problem. But repeated subtraction of rounded numbers can result in catastrophic cancellation.
- 2) According to David Goldberg, "Catastrophic cancellation occurs when the operands are subject to rounding errors." When the cancellation occurs, many of the actually accurate bits/digits may disappear, leaving behind bits/digits contaminated by rounding error.

[David Goldberg's paper](#)

Therefore, the most obvious solution would be to avoid catastrophic cancellation, which means avoiding subtraction of rounded floating point numbers.

To do this, I looked up exponential rules in hopes of finding an equivalent to the negative power. Thankfully, a negative exponent can be represented as:

$$a^{-x} = \frac{1}{a^x}$$

Therefore, all you need to do to avoid the catastrophic cancellation is to compute x^{20} and then do 1 divided by x^{20} to obtain x^{-20}

```
[7]: N = 200 # higher N values don't approximate better
error_list = []

for n in range(1, N+1):

    approx_pos = taylor_exp(n, 20) # compute regular e^20
    approx = 1/approx_pos # get e^-20 by doing 1/e^20
    true_e = math.exp(-20)
    error = (abs(approx - true_e))/(abs(true_e))
    error_list.append(error)

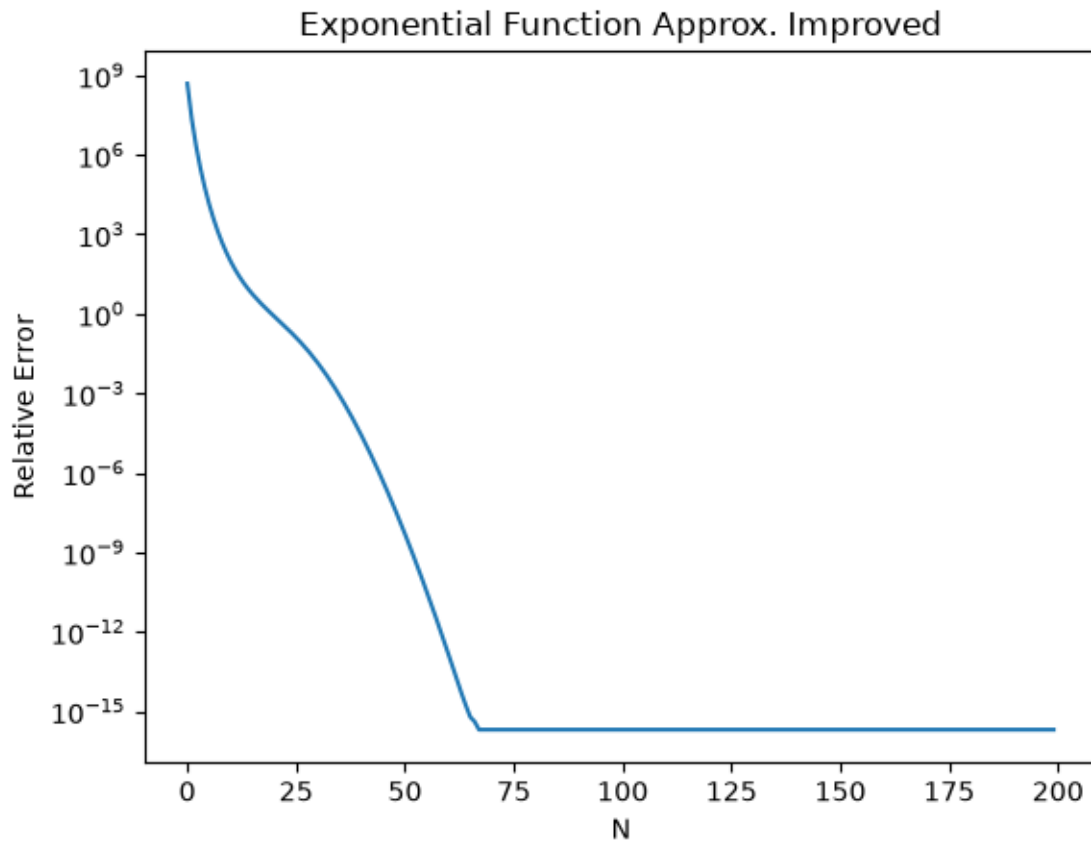
print(min(error_list))
```

2.0065962176423985e-16

As can be observed, the error is now 2×10^{-16} , which is a much closer expected error when compared to the original error of a very large 0.5

```
[8]: plt.plot(list(range(N)), error_list)
plt.semilogy()
plt.title('Exponential Function Approx. Improved')
plt.xlabel('N')
plt.ylabel('Relative Error')
plt.show()

print(f'\nMinimum error obtained was: {min(error_list)}')
```



Minimum error obtained was: 2.0065962176423985e-16

1.1.2 2. (15 points) Matrix-vector multiplication

```
[9]: import numpy as np

print(np.finfo(np.float32).eps) # machine epsilon in float 32
```

1.1920929e-07

```
[10]: A = [
    [1000, 0],
    [0, 0.01]
]

A_inv = np.linalg.inv(A) # compute inverse as done by hand in math notebook
np.linalg.cond(A_inv) # condition number
```

```
[10]: np.float64(100000.0)
```

1.2 Is there a signal in my time-series data? Application of probability theory

1.2.1 Part 2:

Set $N = 1000$. Plot the empirical distribution of (y) for $A \in \{0, 3, 10\}$. That is, for each A , generate many data realizations by calling `generate_data` and compute $\hat{y}$ for each one. Overlay each histogram with the corresponding theoretical Gaussian distribution using the mean and variance you derived above (label the curves clearly). Use these plots to confirm your formulas.

```
[11]: import numpy as np

# provided code in homework PDF
def generate_data(A, N, seed=None):

    """ Generate a noisy time series dataset  $\{t_i, y_i\}_{i=1}^N$ .

    Parameters
    -----
    A : float
        Amplitude of the signal ( $A \geq 0$ )
    N : int
        Number of data points. Time points uniformly sampled from  $[0, 2\pi]$ 
    seed : int or None, optional
        Random seed for reproducibility """

    rng = np.random.default_rng(seed)
    t = np.linspace(0.0, 2*np.pi, N)
    dt = (2*np.pi) / (N - 1)

    s_hat = np.sqrt(dt / np.pi) * np.sin(t) #  $\|s_{\text{hat}}\|_2 = 1$  exactly on this  $\hookrightarrow$  grid
    s = A * s_hat
    n = rng.normal(loc=0.0, scale=1.0, size=N)
    y = s + n

    return t, y, s, n, s_hat, dt
```

```
[12]: import matplotlib.pyplot as plt
from scipy.stats import norm

A_list = [0, 3, 10]
N = 1000
N_sims = 10000 # number of data realizations

bins = np.arange(-4, 14.5, 0.5) # same bin width for every histogram
fig, ax = plt.subplots(figsize=(8, 5))
colors = ["blue", "red", "green"]
```

```

for A, color in zip(A_list, colors):

    samples = []
    for i in range(0, N_sims):

        t, y, s, n, s_hat, dt = generate_data(A, N, seed=i) # seed=i
        p_A = np.dot(y, s_hat)
        samples.append(p_A)

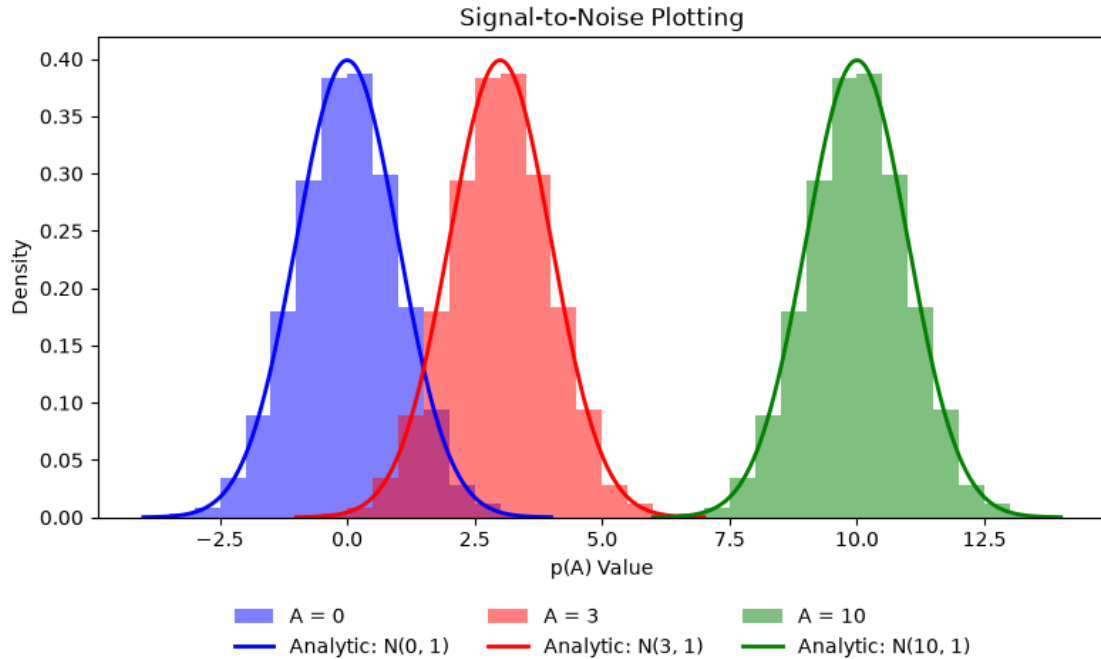
        # computed gaussian has mean = A, std. dev = 1
        x = np.linspace(A - 4, A + 4, 1000) # 4 std. dev around A, 1000 samples
        ↪for smooth curve

    ax.hist(samples, bins=bins, density=True, color=color, alpha=0.5,
            label=f"A = {A}")

    pdf_values = norm.pdf(x, loc=A, scale=1) # where loc is mean and scale is
    ↪std. dev
    ax.plot(x, pdf_values, color=color, lw=2,
            label=f"Analytic: N({A}, 1)")

ax.set_title("Signal-to-Noise Plotting")
ax.set_xlabel("p(A) Value")
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.15), ncol=3,
        ↪frameon=False)
ax.set_ylabel("Density")
plt.tight_layout()
plt.show()

```



As can be observed from the above graph, my Gaussian of theory matches the simulated plots. This is a good sign that my math in the first part of this problem was correct!

1.2.2 Part 3:

(10 points) Now consider two possibilities: no signal ($\mu = 0$) or a signal ($\mu > 0$). From the previous question, observe that for large enough μ , the ($\mu = 0$) distribution is well separated from the ($\mu > 0$) distribution. For example, if $\mu = 10$, setting $\theta = 5$ should separate the no-signal and signal classes nearly perfectly. But for smaller values of μ the distributions of ($\mu = 0$) and ($\mu > 0$) overlap, leading to possible misclassification. There are four outcomes: 1. True positive: You declare “signal present,” and indeed a signal really was present in the data. 2. True negative: You declare “no signal,” and indeed there was no signal in the data (only noise). 3. False positive: You declare “signal present,” but in reality there was no signal (just noise mimicking a signal-like feature). 4. False negative: You declare “no signal,” but there actually was a real signal present.

For the $\mu = 3$ case, empirically choose a value of θ that roughly balances the number of false positives and false negatives for 1000 mock datasets. Present your results by constructing a confusion matrix.

```
[13]: A_list = [0, 3]
      samples_dict = {}

      N_sims = 1000 # 1k per each

      for A in A_list:
          samples = []
```

```

for i in range(0, N_sims):

    t, y, s, n, s_hat, dt = generate_data(A, N, seed=i) # seed=i
    p_A = np.dot(y, s_hat)
    samples.append(p_A)

samples_dict[A] = samples

# sweep over a range from 0 threshold (everything is a signal) to a 1 threshold
↳ (everything is noise)
# where the 0 class is noise and the 1 class is signal

thresholds = np.linspace(0, 3, 101) # don't need to check past 3 because
↳ everything past 3 is a signal anyway and FP/FN will be super unbalanced

print(thresholds)

```

```

[0.  0.03 0.06 0.09 0.12 0.15 0.18 0.21 0.24 0.27 0.3  0.33 0.36 0.39
 0.42 0.45 0.48 0.51 0.54 0.57 0.6  0.63 0.66 0.69 0.72 0.75 0.78 0.81
 0.84 0.87 0.9  0.93 0.96 0.99 1.02 1.05 1.08 1.11 1.14 1.17 1.2  1.23
 1.26 1.29 1.32 1.35 1.38 1.41 1.44 1.47 1.5  1.53 1.56 1.59 1.62 1.65
 1.68 1.71 1.74 1.77 1.8  1.83 1.86 1.89 1.92 1.95 1.98 2.01 2.04 2.07
 2.1  2.13 2.16 2.19 2.22 2.25 2.28 2.31 2.34 2.37 2.4  2.43 2.46 2.49
 2.52 2.55 2.58 2.61 2.64 2.67 2.7  2.73 2.76 2.79 2.82 2.85 2.88 2.91
 2.94 2.97 3.  ]

```

```

[14]: FP = np.array([(samples_dict[0] >= th).sum() for th in thresholds]) #
↳ everything in A_0 is noise (noise called signal)
FN = np.array([(samples_dict[3] < th).sum() for th in thresholds]) #
↳ everything in A_3 is signal (signal called noise)

balance = np.array([abs(fp-fn) for fp, fn in zip(FP, FN)])
print(f'Smallest difference between FP and FN is: {min(balance)} at index: {np.
↳ argmin(balance)}')

```

Smallest difference between FP and FN is: 3 at index: 50

```

[15]: # now just get the corresponding threshold value

best_thresh = thresholds[50]
print(f'Threshold that balances FP and FN is {best_thresh}')

```

Threshold that balances FP and FN is 1.5

```

[16]: from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.metrics import confusion_matrix

```

```

# make predictions
y_pred = []
all_points = samples_dict[0] + samples_dict[3]

y_pred = [0 if p <= best_thresh else 1 for p in all_points] # 0 = noise, 1 =
    ↳ signal
y_true = [0] * len(samples_dict[0]) + [1] * len(samples_dict[3]) # all points
    ↳ in A0 = noise, all points in A3 = signal

cm = confusion_matrix(y_true, y_pred)
cm

```

```

[16]: array([[929,  71],
           [ 68, 932]])

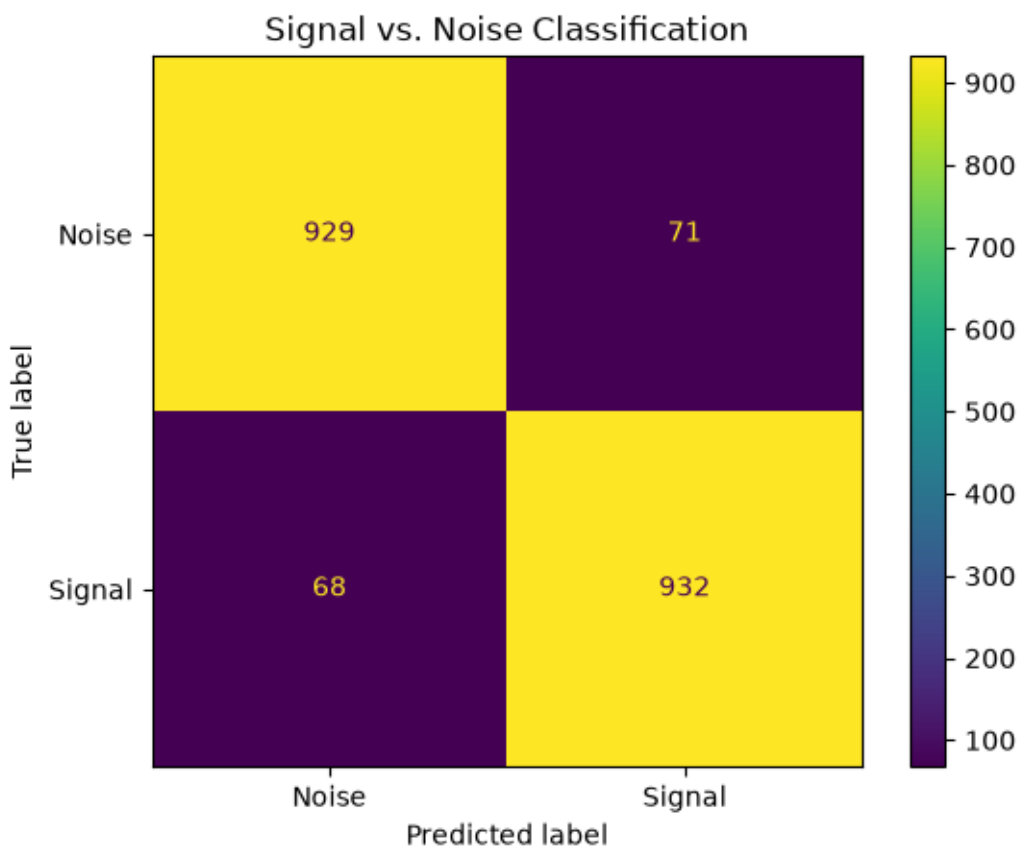
```

```

[17]: # display confusion matrix

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Noise",
    ↳ "Signal"])
disp.plot()
plt.title("Signal vs. Noise Classification")
plt.show()

```



1.2.3